Travaux dirigés Démarrer Laravel et PostgreSQL avec Docker

Objectif

À la fin de ce guide, vous aurez démarré Laravel et PostgreSQL dans Docker, ouvert l'application et connecté pgAdmin à la base de données.

Lisez chaque étape, exécutez les commandes dans l'ordre et comparez votre résultat à la vérification indiquée avant de continuer.

Avant de commencer

Votre poste doit déjà disposer de Docker Desktop, de WSL 2 sous Windows, de Git et de VS Code. Le formateur a préparé ces outils. Démarrez Docker Desktop et attendez qu'il soit opérationnel.

Le premier démarrage peut prendre plusieurs minutes : les images Docker et les dépendances Composer peuvent être téléchargées. En environnement isolé, le formateur doit les avoir préparées et avoir testé la procédure hors ligne.

Les commandes suivantes s'exécutent dans un terminal PowerShell ou WSL. Elles utilisent les conteneurs pour PHP, Composer et PostgreSQL : vous n'avez pas besoin d'installer ces outils séparément sur Windows.

Le projet est disponible dans le dépôt public formation-laravel-special. La branche `starter` contient le point de départ de la formation.

Étape 1 Vérifier Docker et Git

Ouvrez un terminal et saisissez :

```
git --version
docker version
docker compose version
```

À vérifier : Git et Docker Compose affichent leur version. Pour `docker version`, les sections client **et serveur** doivent s'afficher. Si le serveur n'est pas accessible, vérifiez que Docker Desktop est démarré, puis demandez l'aide du formateur si nécessaire.

Étape 2 Récupérer le projet

Dans le dossier où vous souhaitez placer la formation, copiez et exécutez :

```
git clone --branch starter https://github.com/kamel-collab/formation-laravel-special.git app-laravel-postgresql
```

Si le projet vous a déjà été fourni dans un dossier, utilisez ce dossier et passez à l'étape 3.

La commande crée un dossier `app-laravel-postgresql` contenant la branche `starter` du dépôt. Pour entrer dans l'application Laravel :

```
cd app-laravel-postgresql
```

Si vous avez reçu le projet autrement, ouvrez directement le dossier `app-laravel-postgresql` qui contient les fichiers de l'application.

À vérifier : le dossier courant contient notamment `compose.yaml`, `Dockerfile`, `artisan` et `.env.example`. Dans PowerShell, utilisez `Get-ChildItem -Force` ; dans WSL, utilisez `ls -la`.

Ouvrez ce dossier dans VS Code. Vous pouvez garder le terminal intégré de VS Code pour les étapes suivantes. Assurez-vous qu'il est bien placé dans `app-laravel-postgresql` avant chaque commande Docker Compose.

Étape 3 Démarrer les conteneurs

Depuis le dossier `app-laravel-postgresql`, lancez :

```
docker compose up --build -d
```

Cette commande construit l'image de l'application, puis démarre `app`, `db` et `pgadmin`. PostgreSQL initialise sa base dans un volume Docker lors du premier lancement.

Contrôlez les services :

```
docker compose ps
```

À vérifier : `app`, `db` et `pgadmin` sont démarrés. `db` doit indiquer `healthy`. La première fois, `app` peut attendre l'installation des dépendances : c'est prévu. L'application ne sera utilisable dans le navigateur qu'après les étapes suivantes.

Étape 4 Installer les dépendances Laravel

Exécutez Composer **dans le conteneur `app`** :

```
docker compose exec app composer install
```

Composer installe les paquets du projet selon les versions enregistrées dans `composer.lock`. Dans le conteneur, ils se trouvent dans `/var/www/html/vendor`, conservé par le volume Docker `composer_vendor`. Le dossier du projet reste accessible dans VS Code sur Windows ; le dossier `vendor` éventuellement présent sur Windows ne représente pas nécessairement le contenu du volume utilisé par l'application.

Vérifiez l'installation dans le conteneur :

```
docker compose exec app ls -l vendor/autoload.php
```

À vérifier : Composer se termine sans erreur et la commande affiche `vendor/autoload.php`. Le premier téléchargement peut prendre plusieurs minutes.

Après un changement de branche qui modifie `composer.lock`, exécutez à nouveau `docker compose exec app composer install` pour synchroniser les dépendances. Si le volume `composer_vendor` est nouveau ou vide, démarrez les services puis utilisez la même commande.

Étape 5 Créer la configuration locale

Cette étape concerne **une nouvelle copie** du projet qui ne possède pas encore de fichier `.env`.

```
docker compose exec app cp .env.example .env
docker compose exec app php artisan key:generate
```

La première commande crée `.env` à partir du modèle fourni. La seconde génère la clé de l'application dans ce fichier.

À vérifier : la commande de génération indique que la clé a été créée avec succès.

Si `.env` existe déjà, ne relancez pas ces deux commandes. La copie remplacerait votre configuration et une nouvelle clé pourrait rendre inutilisables des données précédemment chiffrées. Demandez au formateur si vous ne savez pas si votre installation a déjà été configurée. Le fichier `.env` est local : ne le partagez pas et ne l'ajoutez pas au dépôt Git.

Étape 6 Ouvrir Laravel

Dans le navigateur, ouvrez :

http://localhost:8088

À vérifier : la page d'accueil Laravel s'affiche. Si le serveur met quelques instants à démarrer après l'installation, attendez puis actualisez la page.

À cette étape, il s'agit du projet Laravel de départ : l'API de documents et la liaison avec l'interface seront développées pendant la formation.

Étape 7 Vérifier Laravel et PostgreSQL

Dans le même terminal :

```
docker compose exec app php artisan --version
docker compose exec app php artisan db:show
```

À vérifier : la première commande affiche Laravel 13. La seconde affiche une connexion PostgreSQL, avec l'hôte `db` et la base `formation_documents`. Au départ, la base peut être vide : les migrations seront étudiées et lancées plus tard avec le formateur.

Dans cette configuration, Laravel utilise le nom `db` pour joindre PostgreSQL sur le réseau Docker. Le port interne PostgreSQL est `5432` ; il n'est pas nécessaire d'ouvrir un port PostgreSQL sur Windows.

Étape 8 Accéder à PostgreSQL avec pgAdmin

pgAdmin est une interface graphique permettant de voir les bases, les tables et leurs données.

1. Ouvrez http://localhost:5050. Le premier démarrage peut prendre quelques instants.
2. Connectez-vous à **pgAdmin** avec l'email `formation@example.com` et le mot de passe `formation_local`.
3. Sur la page d'accueil de pgAdmin, cliquez sur **Add New Server**. Vous pouvez aussi faire un clic droit sur **Servers**, puis choisir **Register > Server...**.
4. Dans l'onglet **General**, saisissez `PostgreSQL formation` comme nom.
5. Dans l'onglet **Connection**, renseignez les paramètres PostgreSQL :

Champ pgAdmin	Valeur
Host name/address	`db`
Port	`5432`
Maintenance database	`formation_documents`
Username	`formation`
Password	`formation_local`

6. Cliquez sur **Save**. Saisissez à nouveau le mot de passe PostgreSQL si pgAdmin le demande.
7. Dépliez **Servers > PostgreSQL formation > Databases > formation_documents > Schemas > public > Tables**.

À vérifier : la base `formation_documents` est accessible. Au départ, la liste des tables peut être vide : les participants exécuteront les migrations plus tard. Si nécessaire, faites un clic droit sur **Tables > Refresh** après une migration.

Le compte pgAdmin et l'utilisateur PostgreSQL ont des rôles différents. Les identifiants ci-dessus sont fictifs et réservés à cette démonstration locale. Le serveur enregistré et les préférences pgAdmin sont conservés dans le volume `pgadmin_data`.

Arrêter et reprendre le travail

Pour arrêter les services à la fin d'une séance :

```
docker compose stop
```

Pour les redémarrer plus tard, dans le même dossier :

```
docker compose up -d
```

Puis ouvrez à nouveau http://localhost:8088. Les données de PostgreSQL restent dans le volume `postgres_data` et les dépendances dans `composer_vendor`. Il n'est pas nécessaire de recréer `.env` ni de régénérer la clé à chaque démarrage.

Ne lancez pas `docker compose down -v` pendant le TD : cette option supprime les volumes du projet, notamment les données de la base et les dépendances installées.

Si un problème apparaît

Situation	Vérification
Docker indique qu'il ne peut pas joindre le moteur	Démarrez Docker Desktop, attendez qu'il soit prêt, puis réessayez.
La commande ne trouve pas `compose.yaml`	Placez-vous dans le dossier du projet `app-laravel-postgresql`, puis réessayez.
Le port 8088 est déjà utilisé	Prévenez le formateur pour identifier le programme concerné et choisir la solution adaptée.
pgAdmin ne s'ouvre pas sur le port 5050	Vérifiez `docker compose ps` et `docker compose logs --tail 30 pgadmin`. Demandez au formateur si le port 5050 est déjà occupé.
pgAdmin ne se connecte pas à PostgreSQL	Vérifiez `db`, le port `5432`, l'utilisateur `formation`, le mot de passe et l'état `healthy` de `db`. Dans pgAdmin, ne remplacez pas `db` par `localhost`.
L'installation Composer échoue lors d'un téléchargement	Vérifiez avec le formateur l'accès réseau ou la préparation hors ligne, puis relancez `docker compose exec app composer install`.
Le navigateur n'affiche pas Laravel	Vérifiez `docker compose ps`, puis consultez `docker compose logs --tail 30 app`. Assurez-vous que les étapes 4 et 5 sont terminées.
Laravel indique que la clé manque	Vérifiez avec le formateur si `.env` existe et si cette nouvelle copie a déjà reçu sa clé. Ne remplacez pas un `.env` existant.
La connexion à PostgreSQL échoue	Vérifiez que `db` est `healthy` avec `docker compose ps` et que la configuration locale provient bien du modèle fourni.

Bilan

- Je sais démarrer et arrêter les services avec Docker Compose.
- Je sais exécuter Composer et Artisan dans le conteneur `app`.
- Je sais ouvrir Laravel sur le port 8088 et vérifier sa connexion à PostgreSQL.
- Je sais ouvrir pgAdmin sur le port 5050 et enregistrer le serveur PostgreSQL.
- Je sais que le volume conserve les données et que `.env` contient la configuration locale.